Does Syntactic Structure Make Language Models More Robust to Noise? LSTM vs. RNNG Subject–Verb Agreement Under Training-Data Corruption

Anonymous ACL submission

Abstract

1 Motivation

The core motivation of this study is to use controlled neural network experiments to investigate a long-standing debate in linguistics and cognitive science: how grammatical knowledge can be acquired from imperfect linguistic input.

According to Chomsky's poverty-of-the-stimulus argument, children are exposed to noisy and incomplete language containing speech errors, disfluencies, and grammatical violations, yet still acquire complex hierarchical grammatical systems. This observation has often been taken as evidence that language acquisition depends on innate linguistic structure rather than solely on statistical learning from input. However, because it is neither ethically nor practically possible to systematically manipulate the linguistic environments of human children, the extent to which grammar acquisition depends on innate structure remains difficult to evaluate through direct experimentation.

Recent advances in computational linguistics provide a new opportunity to address this problem. Neural language models offer a uniquely controllable experimental platform in which both the learner and the learning environment can be manipulated. The inductive biases of different architectures can be interpreted as different forms of internal structure, while the quality of linguistic input can be controlled through the construction of training corpora. By systematically introducing grammatical corruption into the training data, we can simulate increasingly degraded linguistic environments and observe how different learners respond.

Beyond the broader question of whether grammatical knowledge can be acquired from noisy input, this study also seeks to understand how noise affects the internal generalization strategies of language models. Natural language is fundamentally hierarchical, yet standard sequence models lack an explicit inductive bias for hierarchical structure. As grammatical evidence becomes increasingly unreliable, models may continue to rely on hierarchical syntactic representations, or they may gradually abandon them in favor of simpler linear heuristics based on surface word order. Tracking this transition provides a window into how grammatical noise reshapes the internal representations learned by neural language models.

Taken together, these considerations motivate three central questions. First, can language models acquire grammatical rules from systematically corrupted input? Second, does increasing grammatical noise cause models to shift from hierarchical generalizations toward simpler linear strategies? Third, does explicit structural inductive bias improve robustness to noisy linguistic environments? By answering these questions, we aim not only to characterize the robustness of neural language models, but also to contribute evidence to broader debates concerning grammar acquisition, inductive bias, and the learnability of hierarchical structure from imperfect input.

2 Background and Related Work

3 Methods

The design isolates a single comparison: vary only the agreement noise in the training data, holding the test set, vocabulary, training-set size, and architecture fixed, and measure whether a model learns the hierarchical agreement rule (agree with the true subject) or backs off to the linear nearest-noun rule.

3.1 Data and Noise Injection

We build on the subject–verb agreement (SVA) corpus of Linzen et al. (2016), `agr_50_mostcommon_10K` (≈1.58M English

Wikipedia sentences, with rare tokens anonymized so the vocabulary is capped at the 10K most frequent types). Each sentence is annotated with the subject, the inflected verb, the verb's position, and the number of *intervening* nouns that disagree in number with the subject—the "attractors" central to the hierarchical-vs-linear distinction. We train on the real (`orig_sentence`) word forms.

To study robustness to noise we construct five training corpora that are identical except for the rate at which SVA agreement is corrupted. For each sentence, with probability p we flip the verb's grammatical number (singular $\leftrightarrow$ plural), producing an ungrammatical agreement relation while leaving every other lexical and syntactic property unchanged. The flip is a deterministic inflection (`lemminflect`, with hand-specified irregulars for high-frequency copulas and auxiliaries such as *is*/*are*, *was*/*were*, *has*/*have*) applied at the annotated verb index; the *same* function builds the ungrammatical form at evaluation time, so corruption and evaluation are guaranteed consistent. The five noise rates are $p \in \{0, 0.004, 0.02, 0.10, 0.50\}$ (baseline, low, medium-low, medium-high, high). We verified each corpus by confirming that the empirical fraction of corrupted sentences matches p and that the baseline corpus is fully grammatical. **Training-set size is held constant** across conditions, so any change in behavior is attributable to noise rather than data quantity.

3.2 Leak-Free Evaluation Protocol

Because the five corpora are row-aligned with the source corpus, a row index denotes the same sentence in every condition. We reserve a single fixed set of 20,000 held-out row indices, drawn once with a fixed seed (1234), as the test set. These rows are *excluded from every training corpus*, so no test sentence is ever seen in training under any noise condition. The test items are built from the *uncorrupted* source corpus at those same indices: all test sentences are therefore grammatical, and the identical test set scores every condition. Only the training noise varies.

3.3 Minimal Pairs

For each held-out sentence we form a minimal pair: the *prefix* is the sentence up to (but not including) the verb; the *correct* continuation is the original grammatical verb v^+; the *incorrect* continuation is its number-flipped form v^-. A model is scored correct iff it assigns higher conditional probability to the grammatical form,

$$P(v^+ \mid \text{prefix}) > P(v^- \mid \text{prefix}),$$

a single-next-token comparison in the style of BLiMP/SyntaxGym minimal-pair evaluation. We keep only pairs where both verb forms are single in-vocabulary tokens, so the comparison is well defined; this yields **19,819 usable pairs (1,528 with an attractor, 18,291 without)**, identical across all conditions. Each pair is tagged `has_attractor` when at least one intervening noun mismatches the subject in number.

3.4 Models

We compare two architectures trained from scratch on each noise condition. The primary contrast is between a *sequential* model and a *syntactically-biased* model, directly targeting our question of whether an explicit structural bias makes hierarchical agreement more robust to noisy input.

Sequential baseline (LSTM). A word-level LSTM language model in the style of Linzen et al. (2016) and Gulordava et al. (2018): two layers, tied input/output embeddings, and dropout. Each sentence is modeled independently (no streaming across boundaries).

Syntactically-biased model (RNNG). A Recurrent Neural Network Grammar (Dyer et al., 2016), which jointly models structure and terminals through a generative sequence of SHIFT/REDUCE/NT(X) actions and has been shown to improve syntactic generalization over sequential models (Wilcox et al., 2020). We use the `rnng-pytorch` implementation (Noji and Oseki, 2021). RNNG training requires bracketed parse trees; we obtain them once with the Berkeley Neural Parser (Kitaev and Klein, 2018). Crucially, the verb flip changes only the verb *terminal*, not the constituency bracketing, so the parse is identical across all five noise conditions: we parse the clean corpus once and swap the verb terminal per condition, making tree generation a one-time cost rather than $5\times$. We had prepared ON-LSTM (Shen et al., 2019), a softer structural bias requiring no gold trees, as a fallback; it was not needed.

3.5 Metrics and Rule Probing

Our primary metric is minimal-pair accuracy (`acc_all`); as a graded secondary measure we report the surprisal $-\log_2 P(v^+ \mid \text{prefix})$ of the correct verb. To read off *which* rule a model learned,

we split accuracy by attractor presence and report the **attractor gap**

$$\Delta = \text{acc}(\text{no-attractor}) - \text{acc}(\text{attractor}).$$

A small Δ indicates the hierarchical rule (attractors do not hurt); a large Δ indicates the linear nearest-noun rule. Our central analysis is Δ as a function of the training-noise rate.

Cross-model comparability. The LSTM scores all 19,819 in-vocab pairs directly. The RNNG is scored by word-synchronous beam search, which yields per-verb surprisals for only the subset of held-out sentences it can parse within the beam (and does not apply the in-vocabulary verb filter). To keep the LSTM-vs-RNNG contrast on identical items, we evaluate both models on the *matched subset*: the intersection of the pairs the RNNG can score with the LSTM's in-vocabulary pairs. The headline comparison and the H3 test (Section 4) are computed on this matched subset; full reproduction details are in Appendix A.

4 Experimental Setup

4.1 Training

Both architectures are trained from scratch on each noise condition. The vocabulary (10K types) is built once from the clean corpus and shared across every condition and both models, so vocabulary never confounds the comparison.

The LSTM uses embedding and hidden dimension 256, two layers, dropout 0.2, and tied input/output embeddings, optimized with Adam (learning rate 10^{-3}, batch size 64, gradient clipping at 1.0) for 5 epochs over 150K sentences per condition. The RNNG (top-down, fixed-stack) uses word and hidden dimension 256, two layers, dropout 0.3, Adam (learning rate 10^{-3}, batch size 128 with a 15K-token cap), for 8 epochs over the same 150K sentences; per-verb surprisals are obtained by word-synchronous beam search (beam size 100, word-beam 20). Checkpoints are named by condition and seed. A full hyperparameter table is given in Appendix A.

4.2 Conditions and Runs

The grid is 5 noise rates $\times$ 2 architectures $\times$ 3 seeds. The LSTM sweep was run on two machines (local Apple MPS and a Colab T4), giving six runs per condition; the RNNG sweep contributes three seeds per condition. Unless noted, reported values aggregate over seeds, with $\pm$ denoting standard deviation across runs.

4.3 Trajectory Analysis

To ask *when*—not just whether—a rule is acquired, we additionally checkpoint the LSTM every 500 optimizer steps and re-score the full minimal-pair set at each checkpoint, yielding accuracy and attractor-gap trajectories over training (Section 5).

4.4 Hypotheses

We test three preregistered hypotheses:

- **H1.** Overall minimal-pair accuracy decreases as the training noise rate increases.
- **H2.** The attractor gap Δ widens with noise—models lean increasingly on the linear nearest-noun cue.
- **H3.** The RNNG's attractor gap widens *more slowly* than the LSTM's; an explicit structural bias buys robustness to noisy supervision.

H3 is evaluated on the matched subset (Section 3.5). We summarize the per-condition gap difference $\Delta_{\text{LSTM}} - \Delta_{\text{RNNG}}$ with a pair-level bootstrap (resampling the matched pairs, 10^4 replicates) and report the 95% confidence interval; we treat the difference as reliable when this interval excludes zero. This bootstrap reflects item-sampling uncertainty *conditional on the trained models*, while variability due to training is captured by the seed-level mean $\pm$ std, which we report alongside it (with only three seeds, the seed spread is descriptive rather than a formal test).

5 Results

We evaluate every model on the same leak-free set of 19,819 minimal pairs (1,528 with an attractor, 18,291 without), held identical across all five noise conditions. Unless noted, reported values are the mean over six LSTM runs per condition (seeds $\{1, 2, 3\}$, run on both local MPS and Colab T4); $\pm$ denotes standard deviation across runs. Results for the RNNG arm are forthcoming and will be added to Table 1 once training completes.

5.1 Noise degrades agreement, with a threshold near 50%

Overall minimal-pair accuracy is remarkably stable across the first four noise levels and then collapses at the highest rate (Table 1, Figure 1). Accuracy

Condition	Rate	Acc. (all)	Attractor gap
baseline	0.000	0.961 ± 0.006	0.073 ± 0.023
low	0.004	0.958 ± 0.009	0.087 ± 0.030
medium-low	0.020	0.952 ± 0.014	0.103 ± 0.038
medium-high	0.100	0.934 ± 0.020	0.153 ± 0.051
high	0.500	0.708 ± 0.018	0.198 ± 0.033

Table 1: LSTM minimal-pair accuracy and attractor gap by training-noise rate (mean $\pm$ std over six runs). Accuracy is stable through 10% noise and collapses at 50%; the attractor gap grows monotonically throughout.

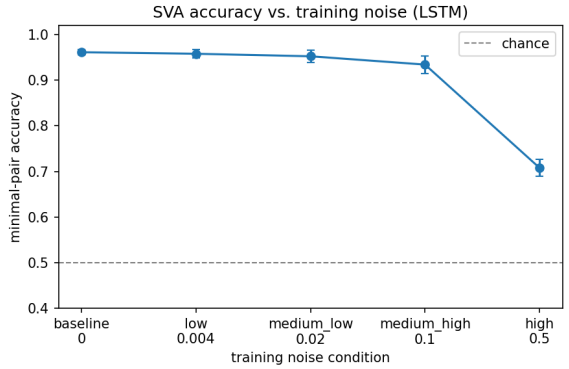

Figure 1: Overall minimal-pair accuracy vs. injected SVA noise rate. The dashed line marks chance (0.5). Performance is flat through 10% noise and falls sharply at 50%.

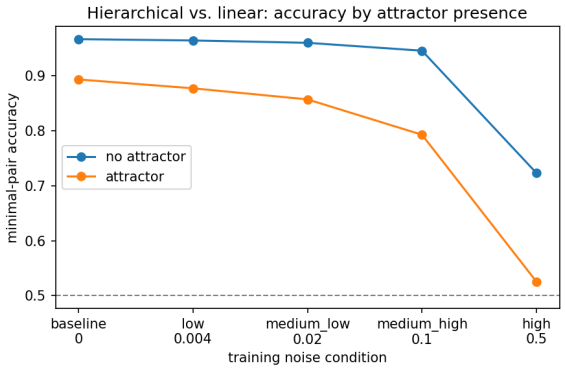

Figure 2: Accuracy on attractor vs. no-attractor items. The two cells diverge as noise rises: attractor accuracy approaches chance while no-attractor accuracy stays high until 50%.

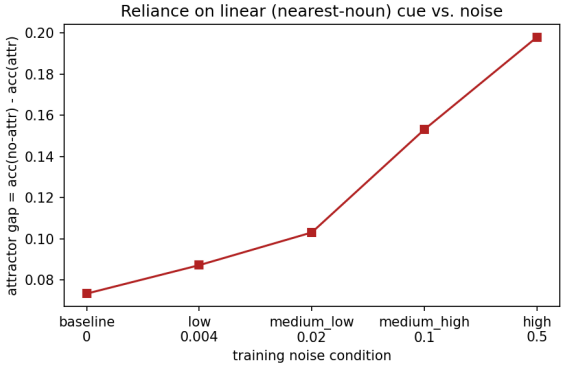

Figure 3: Attractor gap (no-attractor minus attractor accuracy) vs. noise rate. The monotonic rise indicates increasing reliance on the linear cue.

falls only 2.7 points as the error rate climbs from 0% to 10% (0.961 $\rightarrow$ 0.934), but drops 22.6 points between 10% and 50% (0.934 $\rightarrow$ 0.708). The model thus tolerates substantial agreement noise—up to roughly one corrupted verb in ten—before its grasp of the rule breaks down. This non-linear, threshold-like profile matches the first outcome anticipated in our proposal: input quality can be degraded considerably before grammar learning is affected, but beyond a critical point performance falls sharply toward chance.

5.2 Noise pushes the model toward the linear rule

Splitting accuracy by attractor presence isolates *which* rule the model relies on (Figure 2). No-attractor items—where the hierarchical and linear rules make the same prediction—stay above 0.94 until the 50% condition. Attractor items, where the two rules disagree, erode far faster: accuracy falls from 0.894 at baseline to 0.525 (near chance) at 50% noise. As a result the *attractor gap*—our index of reliance on the linear, nearest-noun heuristic—grows monotonically with noise, from 0.073 to 0.198 (Figure 3). Noisier training data drives the model away from the hierarchical rule and toward the linear shortcut, consistent with the "noise pushes models toward simpler generalization" branch of our second predicted outcome.

5.3 Learning trajectories: delay vs. prevention

To ask *when*—not just whether—the rule is acquired, we checkpoint every 500 optimizer steps and re-score the full minimal-pair set at each point (Figure 4). The curves separate the two ways noise can hurt. For the baseline through medium-high conditions, accuracy follows nearly the same trajectory and converges to roughly the same endpoint (0.95–0.97): noise *delays* acquisition but does not prevent it. The 50% condition behaves qualitatively differently—it plateaus near 0.69 and never reaches the rule, i.e. *prevention*. The attractor gap reinforces this: for the clean model the gap shrinks over training as the hierarchical rule consolidates

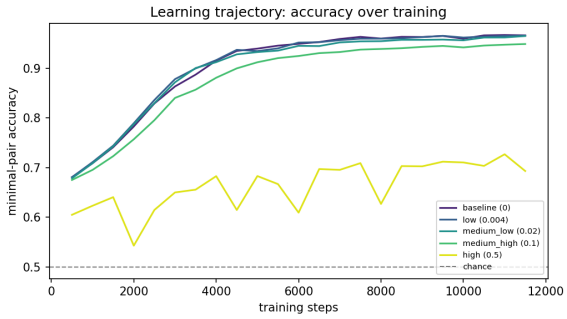

Figure 4: Minimal-pair accuracy over training steps, one curve per noise condition. Low-to-medium noise tracks the clean trajectory and converges (delay); the 50% condition plateaus well below the others (prevention).

(0.148 → 0.054), whereas for the 50% model it *grows* with more training (0.143 → 0.197)—additional exposure to corrupted data entrenches the linear heuristic rather than correcting it.

6 Discussion

When does the rule break? Two findings constrain when subject–verb agreement survives noisy training. First, the breakdown is *threshold-like*, not gradual: accuracy is nearly flat up to a 10% corruption rate and only collapses at 50% (Table 1). Small amounts of ungrammatical input are absorbed, much as a child's grammar is not derailed by occasional speech errors. Second, the attractor gap is a more sensitive early-warning signal than overall accuracy: it climbs steadily (0.073 → 0.198) even while aggregate accuracy looks healthy, revealing that the model is quietly trading the hierarchical rule for the linear nearest-noun heuristic well before its overall performance gives way. The trajectory analysis sharpens the distinction between the two ways noise can act. Up to 10% noise, learning is merely *delayed*—every condition converges to the same accuracy given enough steps—whereas at 50% it is *prevented*: the model plateaus far short of the rule, and further training deepens rather than repairs its reliance on the linear cue.

Does structure buy robustness? Our central architectural question—whether a syntactically biased model (RNNG) withstands noise better than a purely sequential one (LSTM)—requires the RNNG runs to answer, and those are still in progress (Section 5). The LSTM results establish the baseline against which that comparison will be read: a recurrent model with no explicit hierarchical bias acquires the hierarchical rule cleanly under low noise but is pushed toward the linear heuristic as noise grows, and fails outright at 50%. If the RNNG preserves the hierarchical rule at noise levels where the LSTM has already defaulted to the linear shortcut, that would implicate an architectural structural prior as a source of robustness. If both architectures degrade together, it would instead suggest the limiting factor is the statistical quality of the input rather than the model's inductive bias—bearing directly on the poverty-of-the-stimulus framing of Section **??**.

Cognitive-modeling angle. The pattern is suggestive for how structural knowledge might arise from statistical learning. The LSTM does not begin agreement-aware; the hierarchical rule is *acquired* over training, and in the clean setting the attractor gap shrinks as that rule consolidates. Noise does not flip a switch so much as shift the balance between two competing generalizations—hierarchical and linear—both of which the data can support. Heavy enough noise makes the linear rule the better fit to the (corrupted) input, and the model follows the data. That a distributional learner recovers the hierarchical rule at all from this templatic corpus is itself evidence that the structure is more learnable from input than a strong nativist account would predict; that it abandons the rule under sufficient noise marks the limit of that learnability.

Threats to validity. Three caveats bound these claims; we treat them fully in Section 7. Our noise is synthetic and uniform—a fixed per-sentence flip probability—rather than the structured, frequency-dependent errors of natural input. The corpus is templatic and vocabulary-controlled, which aids control but limits external validity. Finally, the trajectory analysis is single-seed; the converging-versus-plateauing pattern is clear, but the precise crossover noise rate should be read as approximate until replicated across seeds.

7 Conclusion

Limitations

References

Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. 2016. Recurrent neural network grammars. In *Proceedings of NAACL-HLT*.

Linnea Evanson, Yair Lakretz, and Jean-Rémi King. 2023. Language acquisition: Do children and language models follow similar learning stages? In

Findings of the Association for Computational Linguistics: ACL 2023, pages 12205–12218.

Kristina Gulordava, Piotr Bojanowski, Edouard Grave, Tal Linzen, and Marco Baroni. 2018. Colorless green recurrent networks dream hierarchically. In *Proceedings of NAACL-HLT*.

Nikita Kitaev and Dan Klein. 2018. Constituency parsing with a self-attentive encoder. In *Proceedings of ACL*.

Adhiguna Kuncoro, Chris Dyer, John Hale, Dani Yogatama, Stephen Clark, and Phil Blunsom. 2018. LSTMs can learn syntax-sensitive dependencies well, but modeling structure makes them better. In *Proceedings of ACL*.

Yair Lakretz, German Kruszewski, Theo Desbordes, Dieuwke Hupkes, Stanislas Dehaene, and Marco Baroni. 2019. The emergence of number and syntax units in LSTM language models. In *Proceedings of NAACL-HLT*, pages 11–20.

Tal Linzen, Emmanuel Dupoux, and Yoav Goldberg. 2016. Assessing the ability of LSTMs to learn syntax-sensitive dependencies. *Transactions of the Association for Computational Linguistics (TACL)*, 4:521–535.

Rebecca Marvin and Tal Linzen. 2018. Targeted syntactic evaluation of language models. In *Proceedings of EMNLP*.

Hiroshi Noji and Yohei Oseki. 2021. Effective batching for recurrent neural network grammars. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*.

Yikang Shen, Shawn Tan, Alessandro Sordoni, and Aaron Courville. 2019. Ordered neurons: Integrating tree structures into recurrent neural networks. In *Proceedings of ICLR*.

Ethan Wilcox, Peng Qian, Richard Futrell, Ryosuke Kohita, Roger Levy, and Miguel Ballesteros. 2020. Structural supervision improves few-shot learning and syntactic generalization in neural language models. In *Proceedings of EMNLP*.

A Reproducibility Details

Pipeline. The LSTM pipeline is `data_utils` → `train` → checkpoint → `evaluate` → `results/eval_results.csv` → `plots`. `data_utils` builds the shared vocabulary and the fixed 20K held-out indices; `minimal_pairs` and `verb_flip` construct the leak-free test pairs from the clean corpus. The RNNG pipeline (`run_rnng_datahub.py`) parses the clean corpus once with the Berkeley Neural Parser, swaps the verb terminal per noise condition, runs the `rnng-pytorch` preprocessing and training, and scores the shared minimal pairs by beam search.

Hyperparameter	LSTM	RNNG
Embedding / word dim	256	256
Hidden dim	256	256
Layers	2	2
Dropout	0.2	0.3
Tied embeddings	yes	—
Optimizer	Adam	Adam
Learning rate	10^{-3}	10^{-3}
Batch size	64	128 (15K-token cap)
Gradient clip	1.0	—
Epochs	5	8
Sentences / condition	150K	150K
Vocabulary	10K (shared)	10K (shared)
Decoding	next-token	beam 100 / word-beam 20

Table 2: Hyperparameters for the LSTM and RNNG. Embedding, hidden size, optimizer, and training-set size are matched; the RNNG additionally consumes benepar parse trees and is decoded by word-synchronous beam search.

Matched-subset comparison (H3). Because the RNNG and LSTM cover slightly different pair sets (Section 3.5), the cross-model comparison is computed on their intersection:

1. `run_rnng_datahub.py --seeds 1 2 3` writes per-pair records (`results/rnng_pairs_*.csv`) and the indices each run could score.

2. `src/match_subset.py` forms the matched index set (RNNG-covered ∩ LSTM-in-vocabulary) and recomputes RNNG metrics on it.

3. `src/evaluate.py --restrict_indices results/matched_idx.json` re-scores each LSTM checkpoint on the same subset.

4. `src/combined_plots.py` produces the side-by-side figures, and `src/stats_h3.py` computes the per-condition gap difference with its bootstrap confidence interval and p-value.

Validity checks. The RNNG run also emits a per-epoch validation perplexity log (`results/rnng_train_ppl.csv`) to confirm convergence, and a parser-quality summary plus a sample of parsed trees (`results/rnng_parse_quality.json`, `rnng_tree_sample.txt`) for hand-inspection, since parser quality on lowercased, anonymized tokens bounds the RNNG's ceiling.

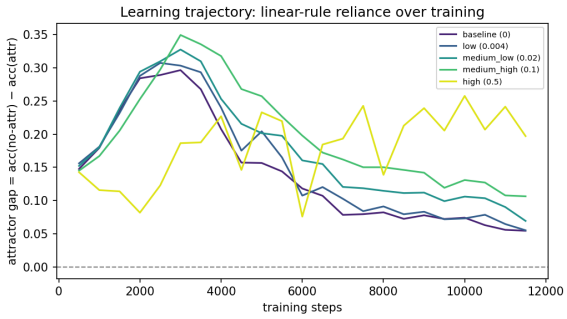

Figure 5: Attractor gap over training steps, one curve per noise condition.

B Additional Figures

Figure 5 shows the attractor-gap trajectory over training (companion to the accuracy trajectory in Section 5): the gap shrinks over training in low-noise conditions as the hierarchical rule consolidates, but grows in the 50% condition as additional exposure entrenches the linear heuristic.